mangen
Программа и результаты испытаний

Агибалов Владислав Сергеевич
НИЯУ МИФИ, ИВТ

2026 год

Аннотация

Документ устанавливает воспроизводимую методику проверки C17-приложения `mangen`, включая unit/CTest, системное вычисление SHA-2, испытания доступа к файлам, многопоточность, sanitizers, утечки и переносимость. Отдельно указаны результаты, сообщённые в журнале запуска macOS, и проверки, не подтверждённые на Linux для полного проекта.

Содержание

1 Цель и критерии приёмки **2**

2 Протоколы выполненных испытаний **2**

2.1 Результаты macOS по журналу 3

3 Структура контролируемых деревьев **3**

3.1 Права доступа 4

4 Методика проверки переносимости **4**

5 Диагностика памяти **5**

6 Подпись и тест отрицательного сценария **5**

7 Отчёт по завершении приёмки **6**

Список литературы **6**

1. Цель и критерии приёмки

Приёмка требует, чтобы выход был однозначен, сортирован и полностью совпадал с независимыми SHA-2, многопоточность не меняла его содержимого, ошибки доступа не скрывались, все ресурсы освобождались и сборка выполнялась на заявленных UNIX-платформах без скачивания зависимостей.

ID	Метод	Условие успеха
V01	CTest со всеми библиотеками	Все запущенные тесты пройдены
V02	`BUILD_TESTING=OFF`	Сборка только C, без GoogleTest
V03	`cmake --install`	Бинарник в `bin/mangen`
V04	Полное доступное дерево	Exit 0, пустой stderr
V05	Дерево с chmod 000	Exit 1, полный доступный набор
V06	Системные SHA-256/384/512	Побайтовое совпадение строк
V07	`-j 1` и `-j 4`	Идентичный stdout
V08	ASan/UBSan/LSan	Нет выявленных нарушений
V09	Компиляторы и ОС	Проходит матрица на macOS/Linux
V10	Подпись и контроль ZIP	Изменение байта отвергается

2. Протоколы выполненных испытаний

В разделе приведены результаты запуска программы на macOS и результаты отдельных проверок сборки на Linux. Числовые итоги на macOS получены из предоставленного журнала команд и их вывода; для Linux объём фактически выполненных операций указан отдельно.

2.1. Результаты macOS по журналу

Измерение	Наблюдение
CTest, обычная сборка	163 из 163 тестов пройдены
CTest, ASan + UBSan	163 из 163 тестов пройдены
Большое дерево с ограничениями	Exit 1; 590 строк stdout; 248 строк диагностики
Системная проверка SHA-256	590 из 590 строк совпали
После восстановления разрешений	6401 запись; exit 0; диагностик нет
Повторная системная проверка	6401 из 6401 строк SHA-256 совпали
Сравнение потоков	stdout с одним и четырьмя потоками идентичен
Запуск ASan/UBSan на полном дереве	Exit 0; 6401 записей; stderr пуст
Apple `leaks`	Сводка: 0 утечек, 0 leaked bytes; есть предупреждение об ограничениях доступа к процессу
Наблюдавшийся пиковый RSS	Около 3.45 MiB в приведённом прогоне

Пределы интерпретации: успешный CTest подтверждает конкретную сборку, компилятор, опции и платформу, а не математическое отсутствие всех гонок; `leaks` сообщил об ограниченном режиме инспекции, поэтому «0 утечек» не является полным доказательством отсутствия всех утечек во всех ветвях.

3. Структура контролируемых деревьев

`scripts/generate_test_trees.sh` создаёт в отдельном системном временном каталоге два дерева. Каждое включает 6400 небольших текстовых файлов с различным содержимым, один бинарный файл 16 MiB, один пустой файл, одну ссылку на регулярный файл и один пустой каталог. Итог для доступного дерева --- 6403 записей. Во втором дереве для файлов и директорий случайно назначаются режимы chmod; не менее одного файла и каталога принудительно остаются недоступными владельцу.

```
TEST_ROOT=$(bash scripts/generate_test_trees.sh)
```

```
bash scripts/verify_test_trees.sh "$TEST_ROOT" \
   ./build/install/bin/mangen
```

Полная проверка включает SHA-256, SHA-384, SHA-512 и режимы 1/4 потока для обоих деревьев, то есть 12 запусков приложения. Выход каждого сравнивается с эталоном на базе системных утилит; проверка не полагается только на количество строк или синтаксис HEX.

3.1. Права доступа

Проверку необходимо запускать от обычного пользователя. В режиме root системные привилегии могут обходить ограничения chmod. Соответствующие утверждения теста в таком окружении пропускаются с явным предупреждением. Успешный частичный запуск должен одновременно удовлетворять трём условиям:

1. код возврата равен 1;
2. stdout содержит в точности все доступные ожидаемые файлы;
3. stderr содержит сообщения об отказах доступа.

Независимая системная `find` также может получить ошибку обхода; это ожидаемо для защищённого дерева, но недопустимо для полного.

4. Методика проверки переносимости

```
bash scripts/test_portability.sh
```

Скрипт определяет текущую ОС, ищет доступные C-компиляторы, отдельно конфигурирует `BUILD_TESTING=OFF` и `ON`, компилирует, выполняет CTest, устанавливает приложение и проверяет его запуск. Отсутствие зависимости C++ в режиме `OFF` является самостоятельным критерием.

Платформа	Основная сборка	CTest	Sanitizers
macOS	По журналу: да	163/163	ASan/UBSan: да
Linux x86_64, библиотеки	C17 синтаксис: да	Не запускался	Не запускались
Linux, полный проект	Не подтверждено	Не подтверждено	Не подтверждено

Дополнительная проверка на Ubuntu 24.04.3 x86_64 с GCC 13.3.0: конфигурация Concurrency Kit 0.7.2 завершилась успешно; проверка `-fsyntax-only` для доступных модулей `matcher`, `queue`, `catalog`, `hash`, `hash_pool` завершилась с кодом 0. Обнаружено одно предупреждение GCC о сравнении, которое всегда ложно, в `queue.c`. Это не линковка и не запуск CTest: полный CMake-цикл не выполнен (CMake отсутствовал в проверочной среде).

Системы macOS и Linux отличаются системными утилитами SHA, некоторыми заголовками и feature-test macros, реализациями `pthread`, правилами линковки и архитектурными компонентами Concurrency Kit. Окончательное утверждение о Linux-поддержке требует исполнения именно в Linux, а не переноса результатов macOS по аналогии.

5. Диагностика памяти

```
bash scripts/test_memory.sh "$TEST_ROOT/clean"
```

Скрипт конфигурирует Debug-сборку с ASan и UBSan, запускает CTest и приложение на полном дереве. На Linux через `ASAN_OPTIONS` включается LeakSanitizer; на macOS при наличии можно дополнительно запускать штатный `leaks`.

Ни один sanitizer не гарантирует нахождение всех ошибок. Для очереди MPMC и её жизненного цикла дополнительно рекомендован ThreadSanitizer там, где он поддерживается данной архитектурой и набором компиляторов.

6. Подпись и тест отрицательного сценария

Создать отдельный ключ подписанта, подготовить trusted_signers со сверенным публичным ключом, подписать манифест и выполнить проверку. Затем изменить один байт в копии манифеста, не меняя подпись: `verify_manifest_signature.sh` должен вернуть ненулевой код. Изменение файла `trusted_signers` или неподписанного манифеста не рассматривается как допустимый способ «исправить» ошибку подписи.

При выпуске ZIP аналогичный отрицательный тест проводится по копии архива: после изменения одного байта `scripts/verify_release.sh` должен завершиться с ошибкой. Проверочный TXT должен быть получен

независимо от недоверенного канала, иначе злоумышленник может заменить и ZIP, и TXT.

7. Отчёт по завершении приёмки

Протокол каждого запуска должен содержать дату, ОС и архитектуру, версии CMake/компилятора, полный Git commit, команды, количество файлов, завершение процесса, stdout/stderr, сводку sanitizer, hash ZIP и тест отрицательного сценария. Неуспешные и недоступные проверки не должны обозначаться пройденными.

Список литературы

[1] GoogleTest. CMake Quickstart. https://google.github.io/googletest/quickstart-cmake.html

[2] LLVM Clang. AddressSanitizer. https://clang.llvm.org/docs/AddressSanitizer.html

[3] LLVM Clang. LeakSanitizer. https://clang.llvm.org/docs/LeakSanitizer.html

[4] Kitware. CTest documentation. https://cmake.org/cmake/help/latest/manual/ctest.1.html

[5] NIST. FIPS 180-4. https://csrc.nist.gov/pubs/fips/180-4/upd1/final